西 安 邮 电 大 学

AI 全栈开发实验

题目：项目总结报告

组　　号：23

组员信息：

学号	姓名	主要分工
2610321101	何朝杨	5.3 Agent 框架集成与联调
2610321103	杜军博	5.1 环境数据、项目总结报告整合与排版
2610321135	王陆洲	5.4 后端服务与接口文档
2610320046	张梓萱	5.4.2 前端页面与浏览器兼容
2610320033	刘凡思雨	5.2 模型选择与 LoRA 微调、宣传海报
2610321168	秦鑫洋	资料归档、截图录屏与交付材料整理

时　　间：2026.11.11

摘要

本项目围绕电商商品图文检索与对话式导购场景，完成了从数据采集、清洗、图文模型微调、推理优化、前后端服务化到 Agent 工具调用的完整工程链路。项目以 9 类商品、9,393 条清洗后的图文数据为基础，采用 Chinese-CLIP ViT-B-16 双塔模型，将图片和中文商品描述映射到同一向量空间，使用 FAISS 完成文本检索、图片检索和图文匹配。实验二在清洗数据上进行 LoRA 微调后，测试集 R@1 从 37.37% 提升到 58.67%，R@5 为 89.08%，R@10 为 96.90%。实验三将微调后的双塔模型导出为 ONNX，并使用 TensorRT FP16 加速，图像塔 batch=1 延迟从 4.59 ms 降至 1.94 ms，延迟降低 57.73%，吞吐提升 136.58%，双塔 R@1 相对同源 FP32 基线下降 0.4283 个百分点。实验五在此基础上以 Qwen3-4B-Instruct-2507 为基座进行 QLoRA 微调，并使用 LangChain 完成工具路由。v2 模型在 10 轮测试上的纯模型工具调用准确率为 90%，加入参数规范化后为 100%。v4 多轮 Agent 在 40 条对话评测中 JSON 解析率 100%、动作准确率 90%、检索命中率 82.5%、安全通过率 100%。项目最终形成了可运行的 React 前端、FastAPI 后端、检索索引、Agent 服务和完整实验凭证。

关键词： Chinese-CLIP；LoRA；FAISS；TensorRT；LangChain；多模态检索；对话式 Agent

项目概述与核心目标

项目承接实验一至实验四的产物，把经过清洗的商品数据和双塔检索模型接入一个可交互的对话式导购系统。系统的核心目标有四项。

1. 统一商品数据和图片路径，建立可以复用的 9 类商品数据集，并保留来源字段与清洗状态。
2. 通过 Chinese-CLIP 双塔模型实现中文文本、商品图片和商品描述之间的相似度检索。
3. 通过 LoRA/QLoRA 微调让小模型学会从自然语言中提取查询意图、商品类目、查询词和数量，并调用白名单工具。
4. 通过 FastAPI、React 和 LangChain 组合出可演示的服务化系统，支持单轮查询、图片检索和多轮会话。

系统总体架构如图 图 1 所示。

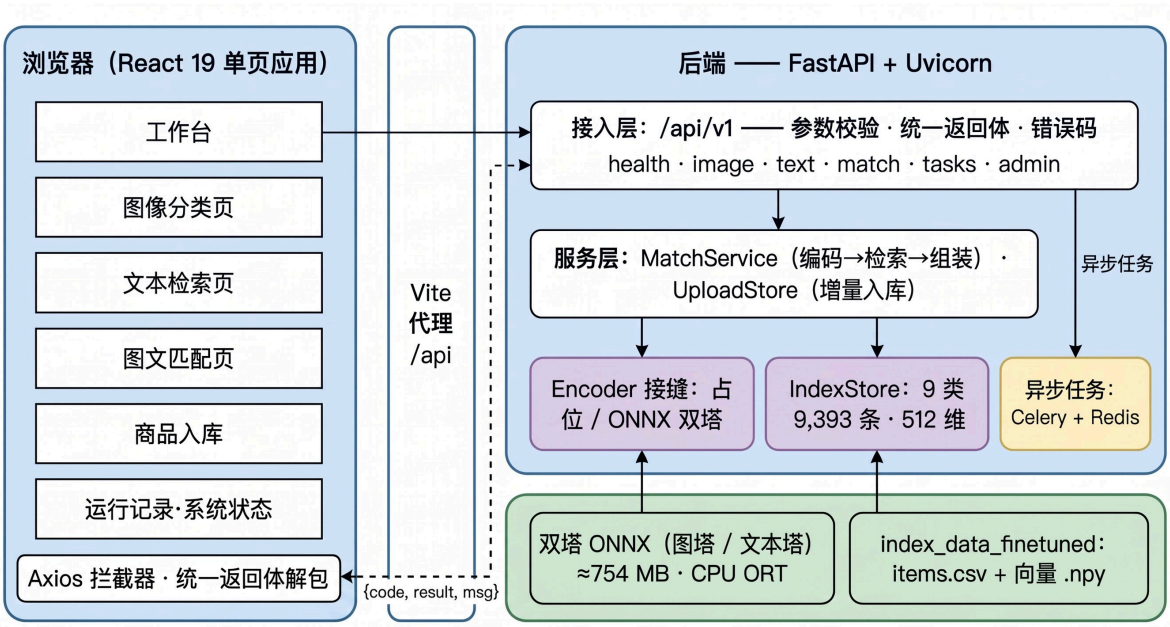


图 1 项目总体架构与数据流

实验 1 至实验 5 的闭环和关键数据流如图 图 2 所示。

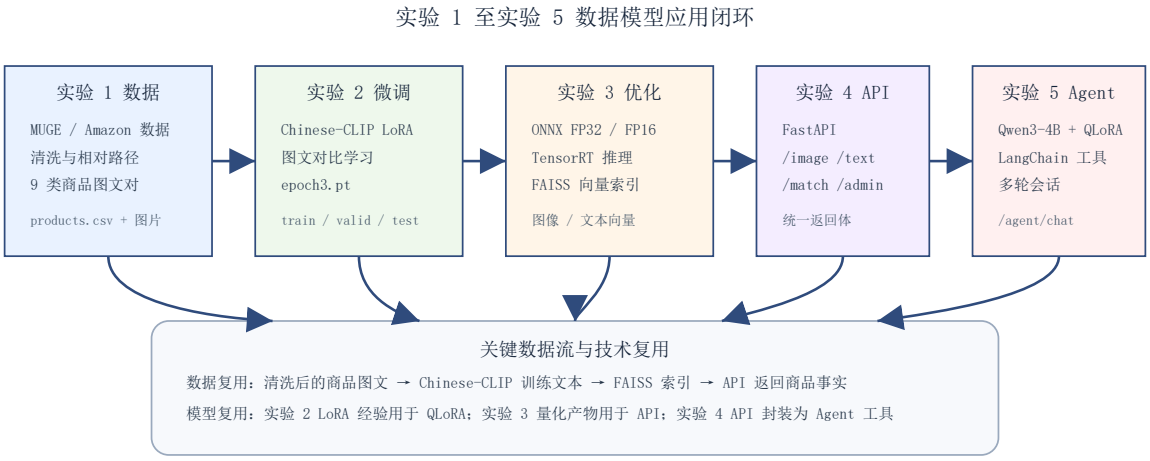


图 2 实验 1 至实验 5 的闭环架构与关键数据流

图 图 2 展示了本项目的完整闭环。实验 1 输出清洗后的商品图文数据，实验 2 使用这些数据微调 Chinese-CLIP，实验 3 将微调权重导出为 ONNX 并进行 TensorRT 优化，实验 4 把图片和文本检索封装为 API，实验 5 再把 API 封装为 Agent 工具。数据流的关键节点是 `products.csv`、商品图片、512 维向量、FAISS 索引和统一的 API 返回体。模型和服务在每个阶段都通过文件或 HTTP 接口交接，避免直接复制业务逻辑。

主要成果与创新点

本项目的主要成果集中在数据、模型和系统三个层面。

1. 数据层完成了 Amazon 与 MUGE 等来源数据的中文化、字段清洗、无关商品剔除、相对路径统一和按商品 ID 防泄漏划分，最终形成 9 类 9,393 条可用于训练和检索的图文数据。
2. 模型层形成了 Chinese-CLIP 微调权重、ONNX FP32/FP16 模型、TensorRT 推理配置，以及 Qwen3-4B Agent 的 v2、v3、v4 适配器。
3. 系统层实现了文本检索、图片检索、图文匹配、商品入库、Agent 对话和会话管理接口，工具调用结果由检索服务提供，模型只负责意图解析和结果点评。
4. 工程层使用相对路径、配置文件和运行凭证目录组织代码，模型和索引可以独立替换，评测结果按 run 归档，便于其他开发者复现。

第一章 项目背景与概述

1.1 项目背景与研究意义

电商平台的商品信息同时包含图片、标题、描述、颜色、材质和商品类目。单纯依赖关键词会受到标题表达差异、属性缺失和同义词的影响，单纯依赖图片又难以表达用户输入的颜色、材质和使用场景。本项目将图像编码、中文文本编码、向量检索和自然语言 Agent 组合起来，让用户可以用文字、图片或连续对话检索商品。

项目采用分阶段交付方式。实验一负责数据建设，实验二负责图文表示学习，实验三负责推理优化，实验四负责 API 和前端，实验五负责 Agent 训练、工具集成与部署联调。每一阶段的输出成为下一阶段的输入，数据、模型和服务均保留可追溯凭证。

1.2 需求分析与应用场景

系统面向三类主要用户操作。用户可以输入“白色运动鞋”等中文查询，上传一张商品图查找相似商品，也可以同时提供图片与文字判断二者是否描述同类商品。对话模式允许用户先指定商品类别，再补充颜色或材质条件。对于模型不可用、工具失败或输入不完整的情况，系统提供规则降级和中文错误提示。

1.3 技术选型依据

商品检索选用 Chinese-CLIP，是因为它能够将中文文本与商品图片编码到同一向量空间，这一设计继承了 CLIP 的跨模态对齐思想[1]，并使用中文预训练模型适配中文商品查询[2]。FAISS 适合在已有向量库上进行快速近邻查询[3]。Agent 侧使用 Qwen3-4B-Instruct-2507 和 QLoRA，以较小显存完成意图解析训练。LangChain 用于组织模型、工具和会话状态。FastAPI 负责稳定暴露 HTTP 接口[4]，React 负责页面交互[5]。

任务书中的 DeepSeek-V4.1-Flash、GLM-5.3 等候选模型规模较大，不适合本组已有显存和预算条件。实际选型改为 Qwen3-4B-Instruct-2507，使用 4-bit NF4 量化与 LoRA 适配器，保持了训练可行性和本地演示能力。

1.4 项目目标与预期成果

任务书要求 Agent 工具调用准确率达到 90%，响应时间控制在 5 秒以内，并完成前后端联调。本项目已完成 5.1、5.2、5.3，5.4 后端和前端核心代码已经入库。对话 Agent 的模型评测和多轮逻辑已有凭证，前端完整回归仍需在最终交付前按照第八章清单执行。

第二章 实验环境与技术栈

2.1 硬件环境配置

实验环境与数据规模见表 1，训练机与演示机分开记录，避免将训练耗时和部署响应时间混为同一指标。

实验硬件与数据规模如表 表 1 所示。

项目	配置
数据与索引	9 类、9,393 条商品；FAISS 512 维向量索引
训练	AutoDL RTX 4090 48GB vGPU，3 epoch，57.53 秒
Agent 验证	RTX 3080Ti 12GB 与 RTX 5060 8GB，v2 纯模型 90%
部署开发	Windows 11，Python 3.12；本地 CPU/GPU 服务可切换

表 1 实验硬件与数据规模

2.2 软件与开发环境

项目使用 Python、PyTorch、Transformers、PEFT、Accelerate、FastAPI、Uvicorn、React、Vite、LangChain、FAISS 和 TensorRT。图表中的技术栈见表 2。检索后端支持 ONNX Runtime CPU/CUDA/TensorRT，Agent 服务支持本地 Qwen3-4B LoRA 适配器，也保留远程 OpenAI 兼容接口的接入方式。仓库内的路径均以项目根目录为基准，模型文件和 `.env` 等机器相关文件按说明放置。

2.3 主要技术框架与工具

系统技术栈分层如表 表 2 所示。

层次	组件与作用
数据层	processed_data_cleaning、属性字段、商品图片、相对路径
模型层	Chinese-CLIP 双塔、Qwen3-4B、LoRA/QLoRA
检索层	FAISS、向量归一化、文本/图片/融合排序
服务层	FastAPI、Uvicorn、LangChain Agent、会话状态
前端层	React、Ant Design、Axios、Vite

表 2 系统技术栈

2.4 版本管理与协作工具

项目使用 Git 管理代码、脚本、文档和可复现实验凭证。大模型权重、ONNX 文件和本地密钥不作为普通源码处理。每次训练或评测以独立 run 目录保存输入快照、配置、结果表、图表和说明文档，避免把不同模型或不同索引的结果混合。

第三章 数据采集与预处理

3.1 数据需求分析

商品数据需要同时提供商品 ID、类别、图片路径、标题或描述，以及可用于构建检索文本的颜色、材质和细分类。价格等不能可靠识别的字段允许为空。清洗过程优先保证类别正确、图片可读和路径可追溯，再补充颜色、材质和描述。

3.2 数据采集与收集

数据来源包括 MUGE 和 Amazon 译文数据。采集后先保留原始 CSV 与图片目录，再由分类脚本按商品类别整理到 `data/processed_data/`，经过属性清洗和人工核验后形成 `data/processed_data_cleaning/`。图片字段统一使用相对项目根目录的路径，避免在不同机器之间失效。

3.3 数据预处理流程

两类来源数据的类目构成与样本分布如图 图 3 和图 图 4 所示。9 个类别的样本量并不均衡，鞋类与服装类样本最多，厨房用品和家居用品相对较少；这一分布决定了微调时各类别的学习机会，也是后文按类别分析检索效果时的参照。所有样本在入库前都经过图片可读性校验，坏图与重复图在清洗日志中留痕。

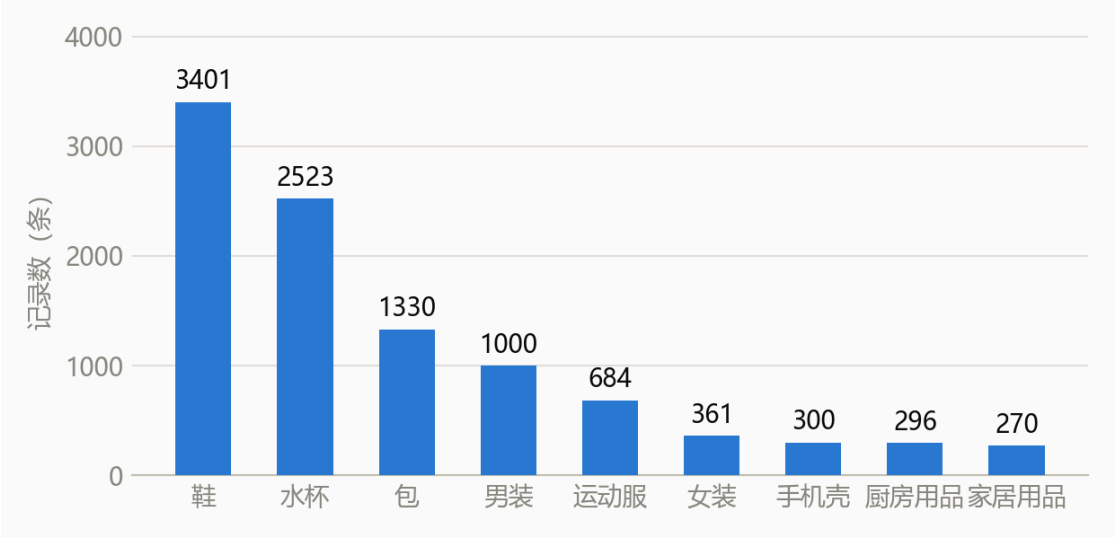


图 3 数据来源与类目构成

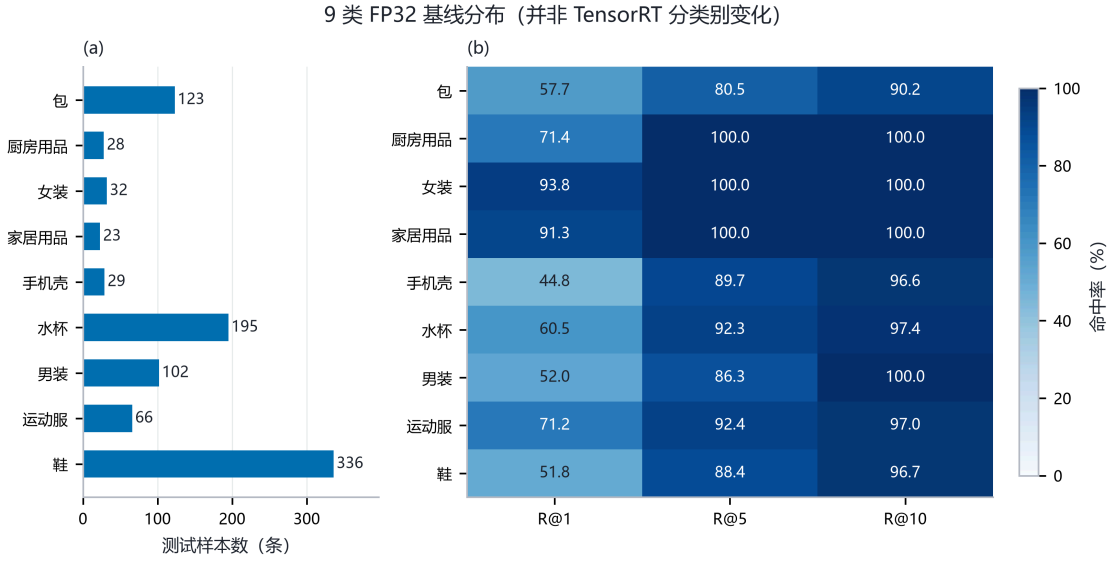


图 4 九类别样本分布与零样本基线

处理流程为：原始数据读取 → 类别整理 → 图片存在性检查 → 非本类商品剔除 → 颜色、材质和细分类补全 → 商品描述模板化 → 按 item_id 哈希切分 train/valid/test → 生成 Chinese-CLIP 文本 → 建立 FAISS 索引。每一步都保留 CSV 和 JSONL 日志，清洗脚本写入 is_read 标志，后续增量加入的数据可跳过已读记录。

3.4 数据清洗与增强

清洗脚本对每个类别单独处理，利用商品图片和原有文本识别类目、颜色、材质和细分类。模型输出经过字段校验和空值回退，非法类别不会直接进入最终数据。描述使用统一模板“商品大类；细分类；颜色；材质”，既方便人工审核，也让训练和索引文本保持一致。果冻、书架等与当前类别无关的商品被剔除并记录在清洗报告中。

3.5 数据集建设

训练集和验证集的商品划分如表 表 3 所示。

划分	样本数	依据	说明
train	7,520	sha256(item_id)	用于模型微调
valid	939	sha256(item_id)	用于超参与早停判断
test	934	sha256(item_id)	只在最终报告使用

表 3 图文训练集划分

训练集划分结果见表 3。按照 item_id 哈希切分可以保证同一个商品的多张图片不会跨越训练集、验证集和测试集。训练和测试文本使用同一个 build_product_text 模板，避免训练文本和索引文本分布不一致。

第四章 模型设计与训练

4.1 模型选型分析

Chinese-CLIP 使用视觉塔和文本塔分别编码图片与中文文本，再通过对比学习将同一商品的两个模态拉近[2]。Agent 侧选择 Qwen3-4B-Instruct-2507，采用 QLoRA 4-bit 量化降低显存占用。两类模型承担不同职责，Chinese-CLIP 负责相似度表示，Qwen 负责结构化意图和工具参数，避免让语言模型直接编造商品事实。

4.2 模型架构设计

Chinese-CLIP 的图像塔输入 224×224 图片，文本塔输入中文商品模板，输出 512 维向量并进行 L2 归一化。FAISS 对索引向量建立内积索引，归一化后的内积等价于余弦相似度[3]。Agent 的四工具白名单为 text_search、image_search、list_categories 和 get_item_detail。检索结果由工具返回，再作为 检索结果： 内容回喂给模型生成点评。

Agent 微调与推理流水线如图 图 5 所示。

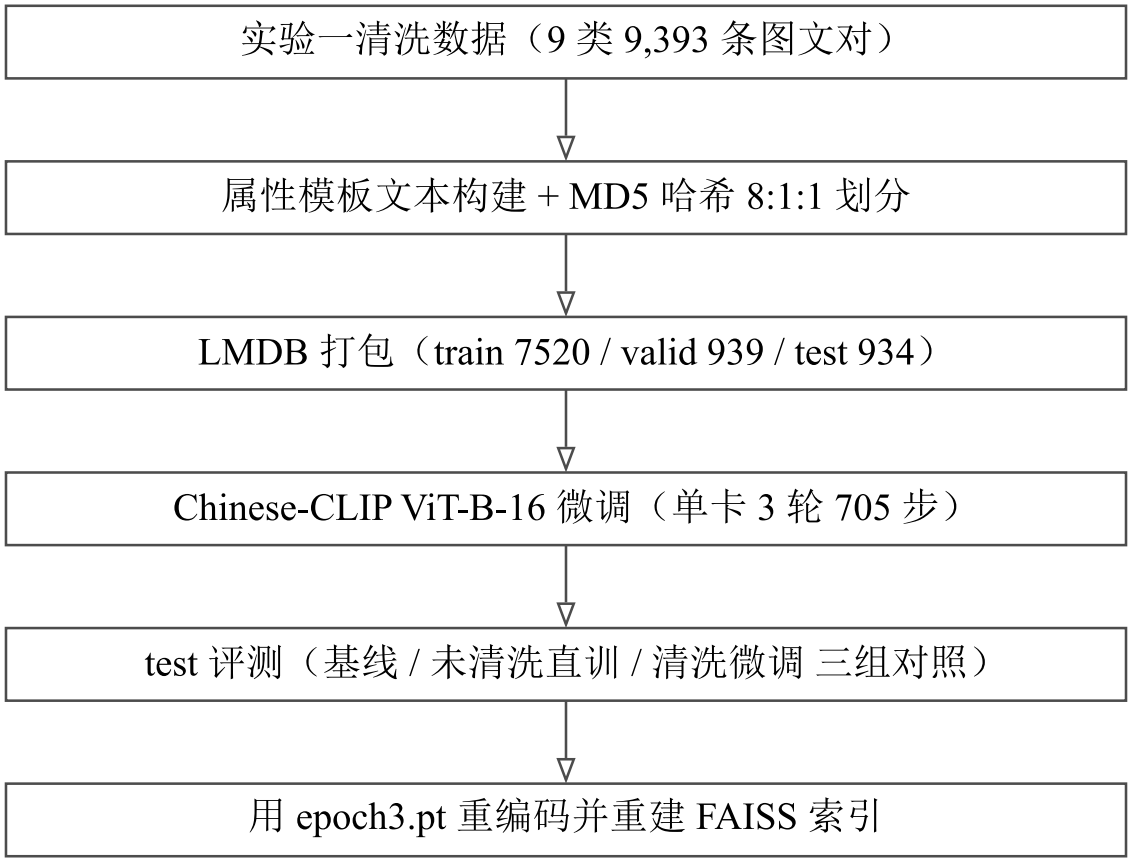


图 5 Agent 微调与推理流水线

4.3 训练策略与配置

5.2 使用 QLoRA 4-bit NF4 和 double quant，LoRA rank 为 16，alpha 为 32，dropout 为 0.05。LoRA 通过低秩适配器减少可训练参数[6]，覆盖 q、k、v、o 和 gate、up、down 投影。训练数据按 `sha256(item_id)` 划分，SFT 使用 89 条训练样本，11 条验证样本，训练 3 epoch、18 个优化步。训练机为 AutoDL RTX 4090 48GB vGPU，训练耗时 57.53 秒，总成本约 0.13 元。

4.4 模型训练过程

初始 v1 数据中工具调用样本只有 15 条，模型容易把 `text_search` 写成 `tool_search`，纯模型准确率只有 10%。随后扩充到 78 条工具调用样本，去掉输入中写死的类目提示，让模型学习从商品词推断类目，得到 v2 适配器。v2 在 10 轮评测中纯模型工具调用准确率达到 90%，加入参数规范化后达到 100%。v3 和 v4 进一步加入四工具动作协议、多轮上下文和检索结果回喂。

微调过程的训练损失与验证准确率曲线如图 图 6 所示。3 个 epoch 内训练损失持续下降，验证集总体准确率从 12.88% 提升到 75.76%；由于验证集规模小，曲线波动较大，训练轮数不再增加以避免过拟合风险。

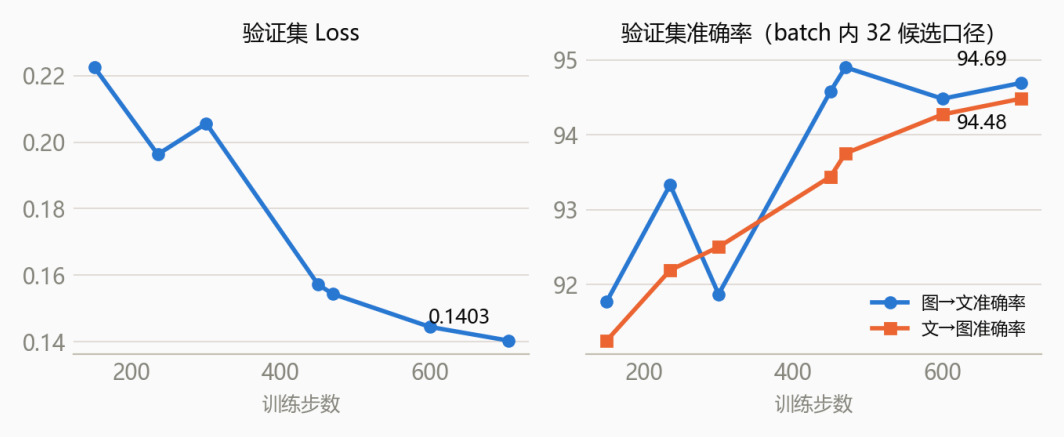


图 6 QLoRA 微调训练曲线

5.2 验证集微调前后结果如表 表 4 所示。

指标	基座	LoRA v1	变化
attribute	28.33%	100.00%	+71.67pp
intent	0.00%	66.67%	+66.67pp
tool_call	0.00%	0.00%	n=1，样本过少
总体	12.88%	75.76%	+62.88pp

表 4 5.2 验证集微调前后对比

第五章 算法优化

5.1 性能评估基准

实验二的微调结果见表 4，实验二在 934 条 test 图文对上评估图文检索。预训练零样本基线 R@1 为 37.37%，清洗数据微调后 R@1 为 58.67%，R@5 为 89.08%，R@10 为 96.90%。实验三以同源 FP32 基线重新复算，R@1 为 58.5653%，用于 ONNX 和 TensorRT 的一致性比较。

双塔微调在测试集上的精度对比与召回曲线分别如图 图 7 和图 图 8 所示。清洗数据微调相比零样本基线的提升集中在 R@1 与 R@5 档位，说明微调主要改善的是排序头部质量；R@10 已达 96.90%，候选池尾部接近饱和。

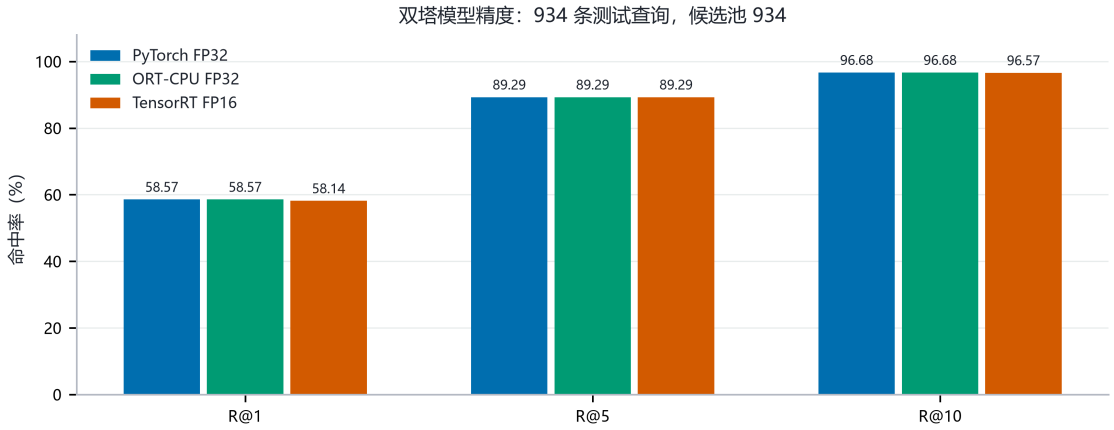


图 7 双塔检索精度：零样本基线与微调对比

清洗与未清洗数据的对照实验见 9.3 节。召回率随候选数变化的曲线如图 图 8 所示。

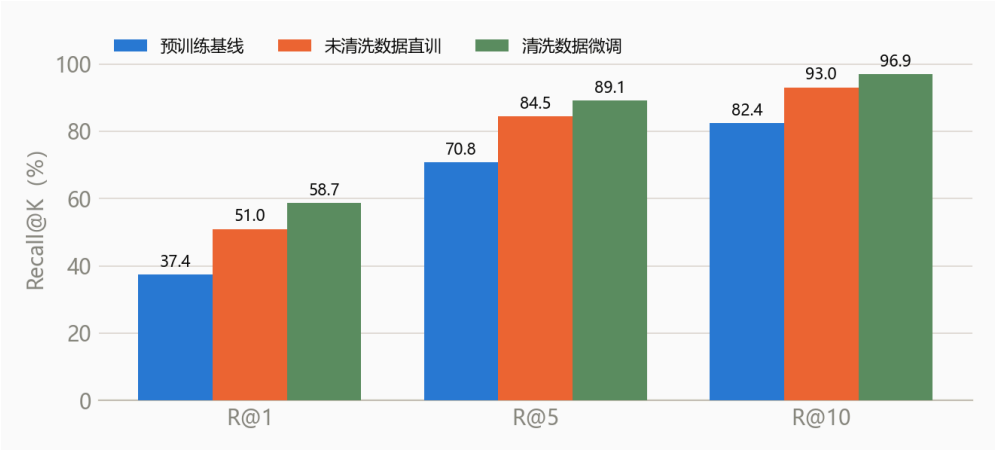


图 8 图文检索召回曲线

5.2 推理优化一：双塔导出

模型先导出为 ONNX FP32，再进行图层级 FP16 转换。ONNX 作为跨框架模型交换格式[7]，便于将训练代码与部署代码解耦。这样可以避开直接将 PyTorch 模型 `.half()` 导出时 LayerNorm 混合精度错误。图像塔固定 224×224，文本塔保留 batch 和序列长度动态维度。推理侧使用同一批候选和同一测试集，确保优化结果可比较。

5.3 推理优化二：TensorRT

ONNX Runtime CPU 在同线程配置下相比 PyTorch CPU 没有实质加速，ORT-CUDA 也受内存拷贝和算子回退影响。TensorRT FP16 通过算子融合和 kernel 重写取得稳定收益。图像塔 batch=1 加速比 2.366×，batch=32 加速比 2.115×，两档延迟降低和吞吐提升均达到任务书阈值。

推理加速的完整流水线（导出、转换、引擎构建、配对测速）如图 图 9 所示。



图 9 ONNX 与 TensorRT 推理加速流水线

不同推理后端的延迟与加速比如表 表 5 所示。

后端	batch	基线 ms	优化 ms	加速比
ORT-CPU FP32	1	90.56	85.24	1.062×
ORT-CUDA FP16	1	4.98	4.54	1.097×
TensorRT FP16	1	4.59	1.94	2.366×
TensorRT FP16	32	50.43	23.84	2.115×

表 5 图像塔推理延迟与加速比

延迟与吞吐的可视化对比分别如图 图 10 和图 图 11 所示。两档 batch 下的加速比稳定在 2 倍以上，收益来自 TensorRT 的算子融合与 FP16 kernel 重写，而非简单的半精度转换。

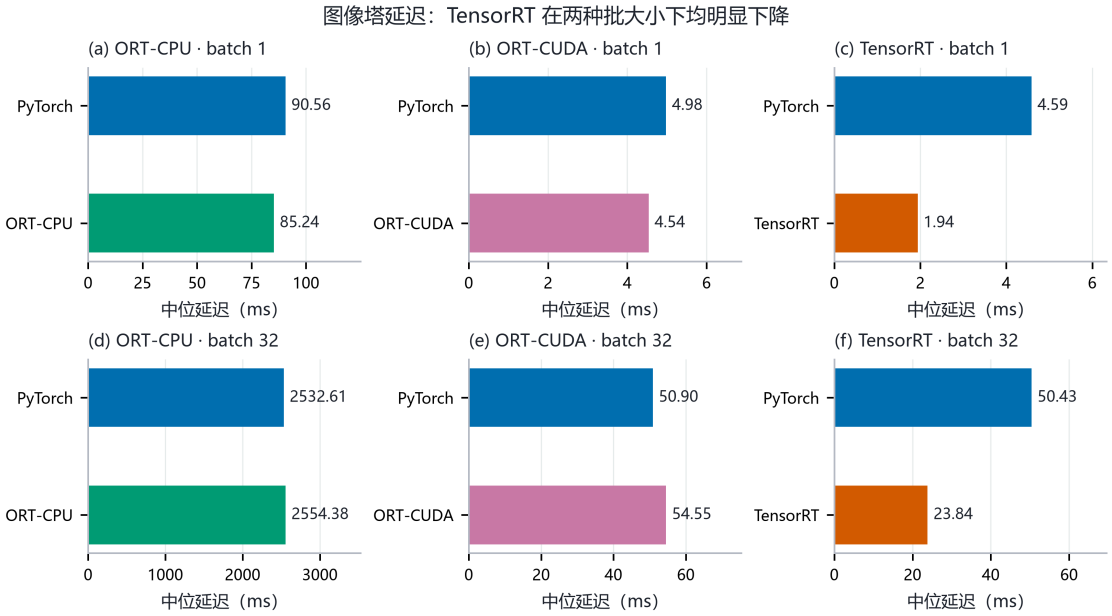


图 10 图像塔推理延迟对比（batch=1 与 batch=32）

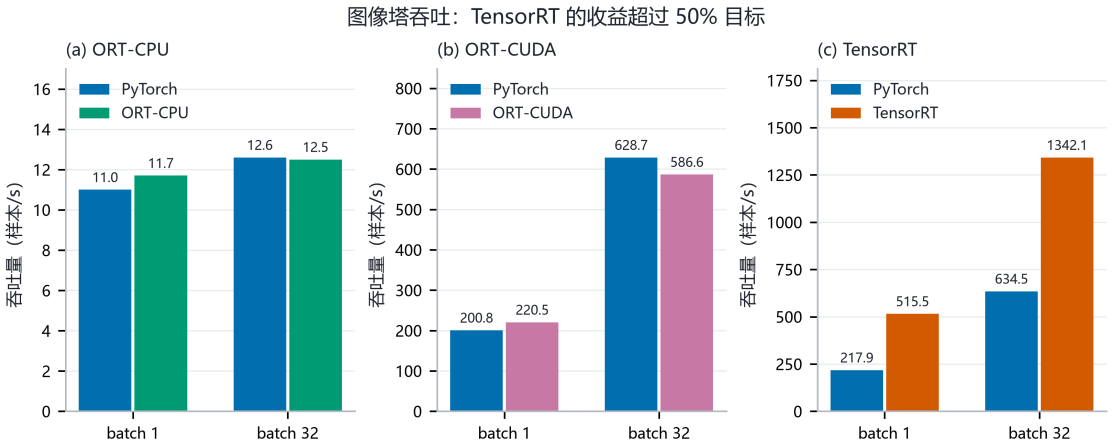


图 11 图像塔推理吞吐对比

精度代价方面，FP16 与 FP32 在 934 条测试查询上的逐后端误差如图 图 12 所示，TensorRT FP16 的双塔 R@1 净差为 0.4283pp，远小于任务书 1pp 的上限。量化后模型文件体积变化如图 图 13 所示，FP16 相比 FP32 显著缩小，便于部署分发。

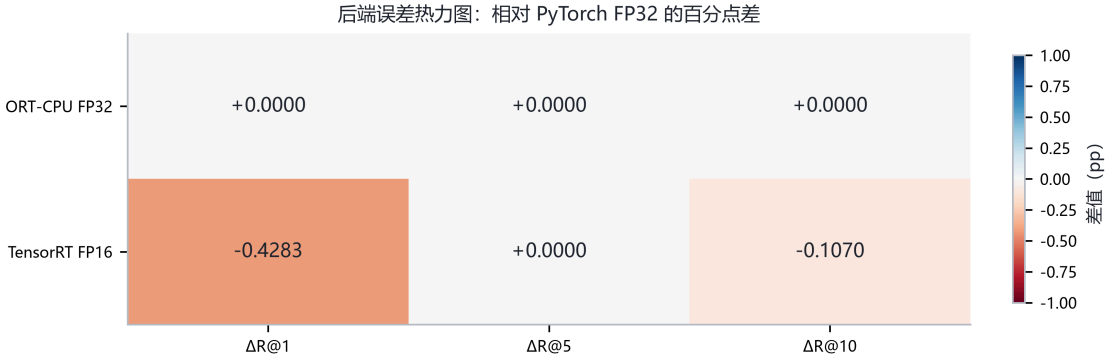


图 12 各推理后端精度误差热力图

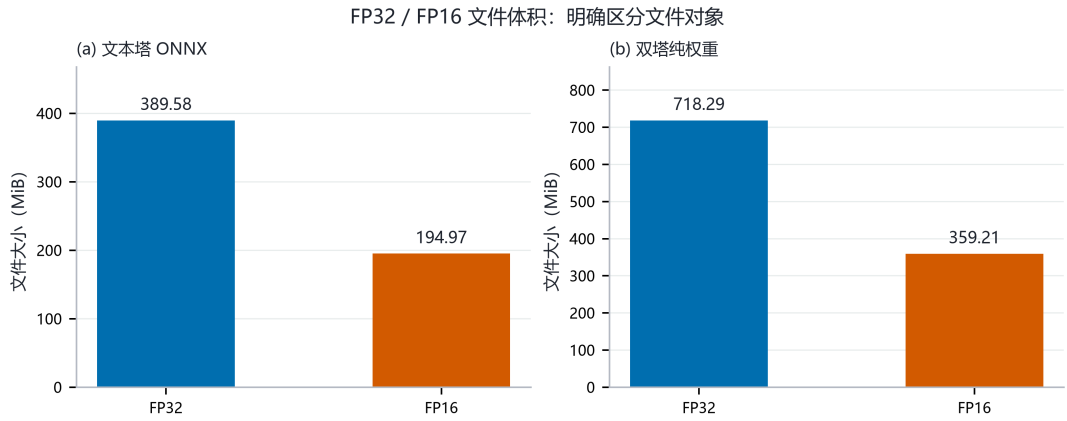


图 13 不同精度模型的文件大小对比

5.4 Agent 推理优化

Agent 侧的优化重点是减少模型输出格式错误和工具参数错误。运行时使用工具白名单和编辑距离纠错，将 `tool_search` 对齐为 `text_search`，再对类别、数量和查询词做规范化。多轮模式保存最近 8 条消息、最多 6 句摘要和当前类目槽位。上一轮查询为“运动鞋”、本轮输入为“白色”时，后端合并为“运动鞋 白色”，避免第二轮查询过于宽泛。

5.5 算法优化测试与验证

不同 Agent 版本的评测结果如表 表 6 所示。

评测版本	数据	JSON	动作	检索命中
v2 纯模型	10 轮	100%	90%	—
v2 加后处理	10 轮	100%	100%	—
v3 单轮	40 条	95%	92.5%	82.5%
v4 多轮	40 条	100%	90.0%	82.5%

表 6 Agent 评测结果

第六章 前后端开发

6.1 系统架构设计

系统采用 React 单页应用、FastAPI 接入层、服务层和检索索引四层结构。React 和 FastAPI 的职责分别见表 2。前端请求通过 Axios 发送到 `/api/v1`，后端校验请求后调用图像塔或文本塔，返回统一的 `{code, result, msg}`。Agent 路由额外返回 `reply`、`thinking`、`tool_calls` 和 `items`。

6.2 前端开发

前端复用实验四 React 工程，增加 AgentChat 页面和路由 `/agent`。页面支持自然语言输入、会话 ID、图片粘贴或拖拽、结果卡片、工具调用过程和清空会话。用户可以在同一个页面完成文本检索和图片相似商品推荐。

前端系统首页如图 图 14 所示。

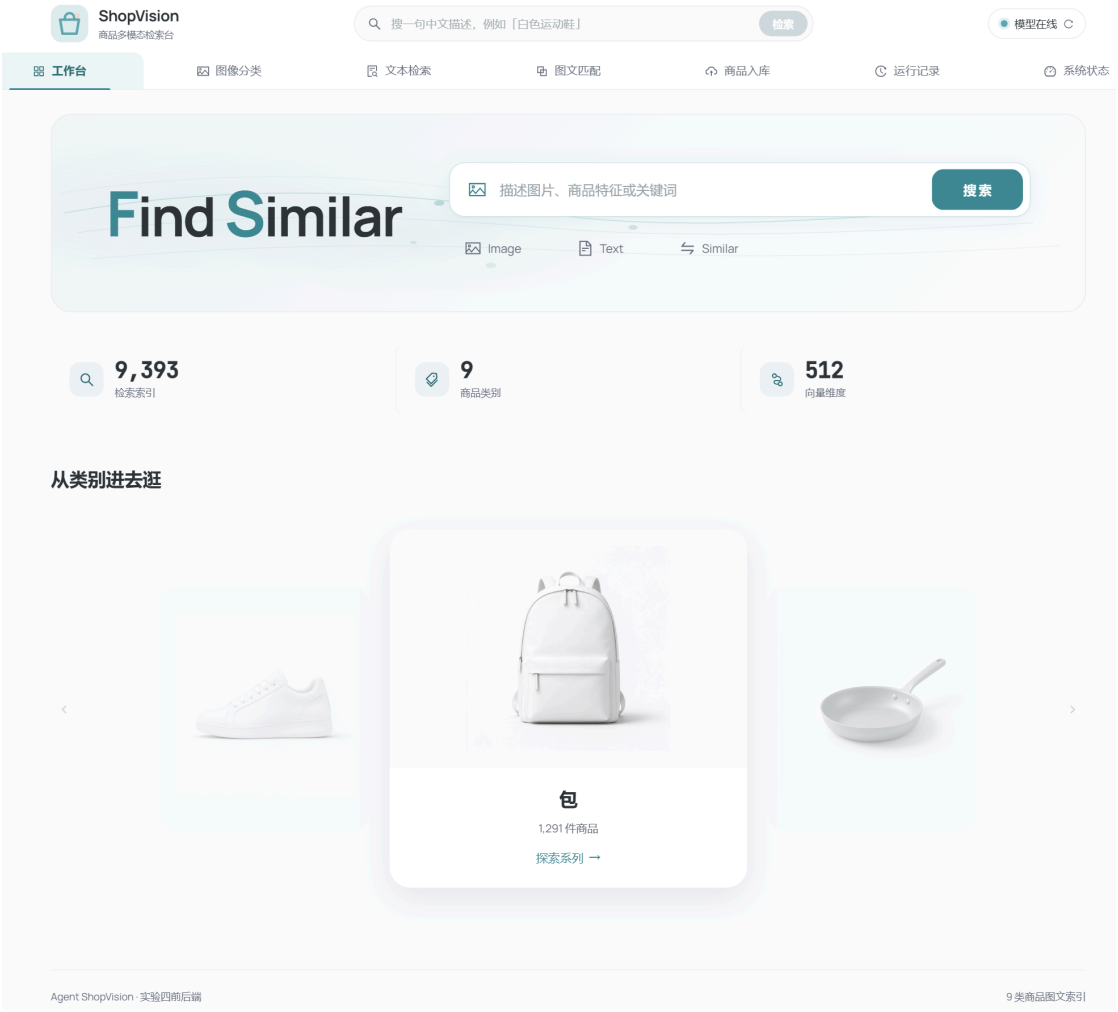


图 14 前端系统首页

文本检索与图文匹配两个核心页面的运行效果分别如图 图 15 和图 图 16 所示。文本检索页展示命中类别分布与按相似度排序的 Top-K 结果；图文匹配页将图像与文本两条链路的判定并排呈现，用户可以直接对比两类判定是否一致。

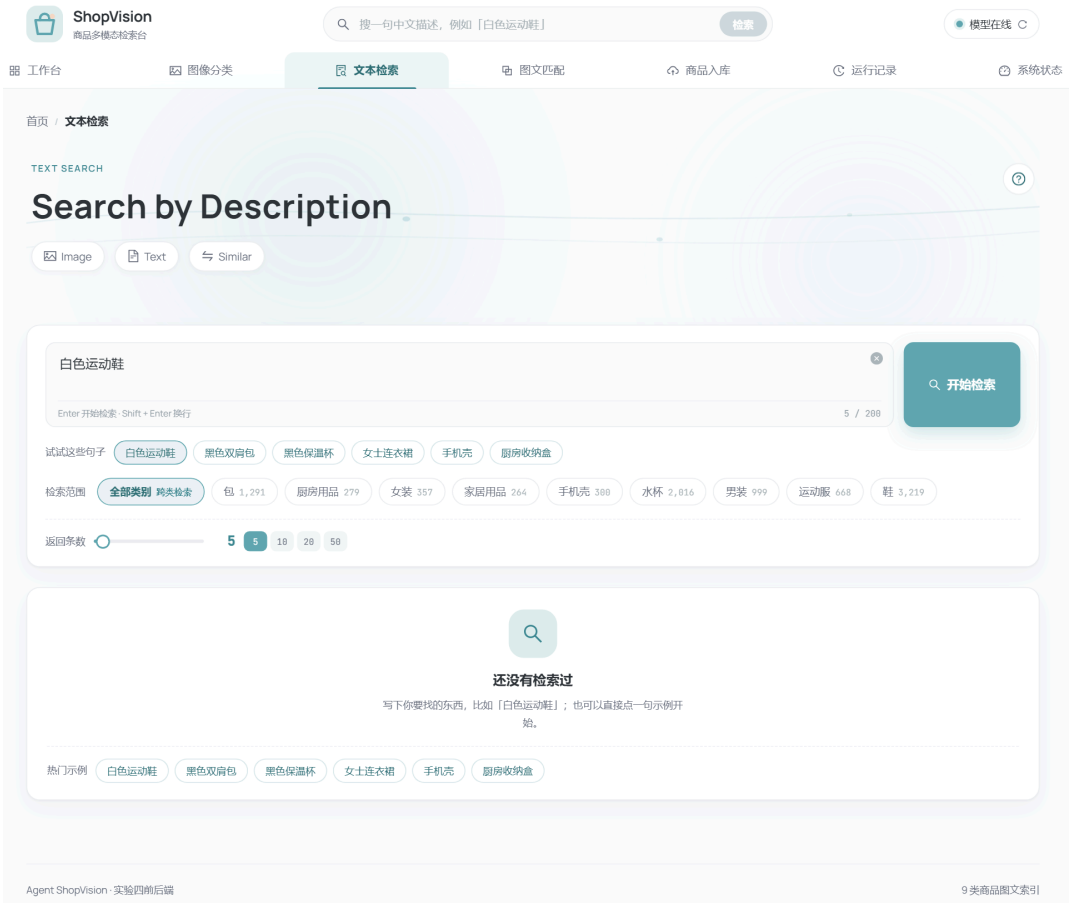


图 15 文本检索页运行效果

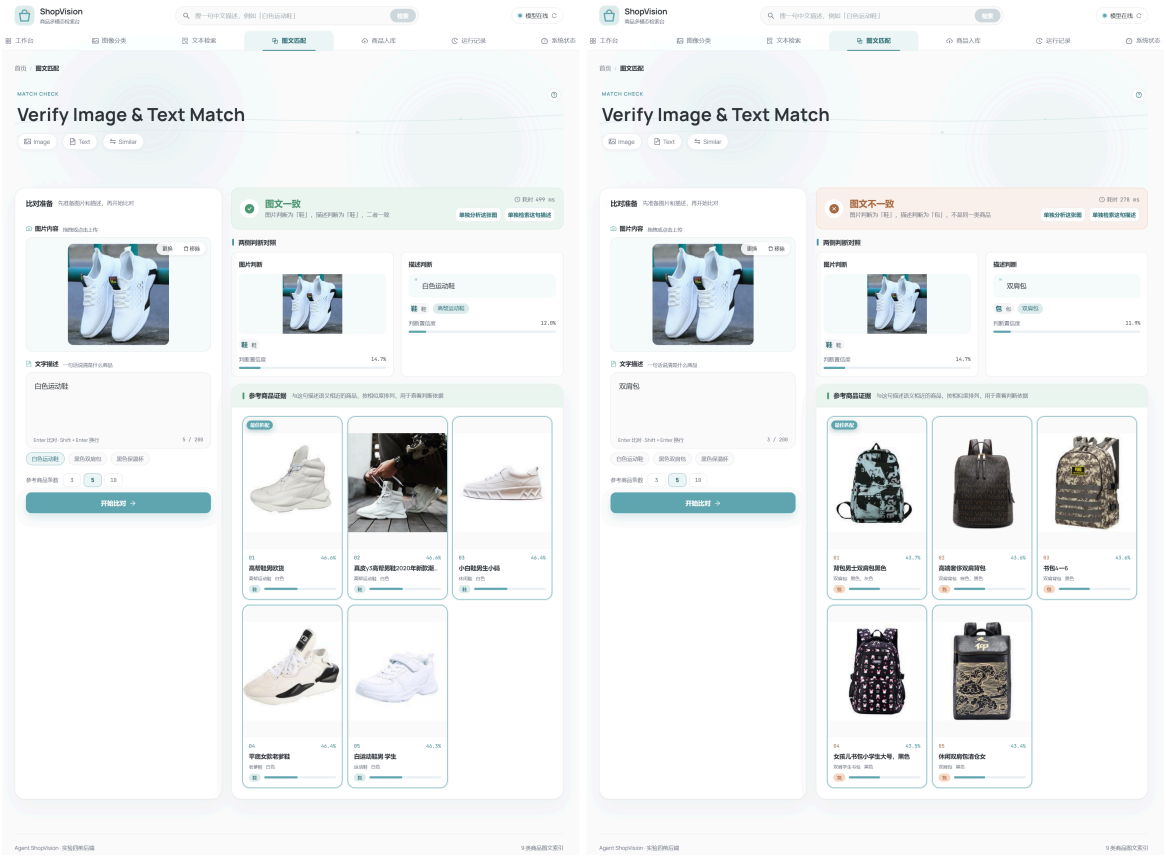


图 16 图文匹配页：图与文判定并排对比

输入不完整、模型后端不可用等异常场景的中文提示如图 图 17 所示，错误码遵循统一的 40000/40001/50001 语义，前端按码表映射为可读文案。

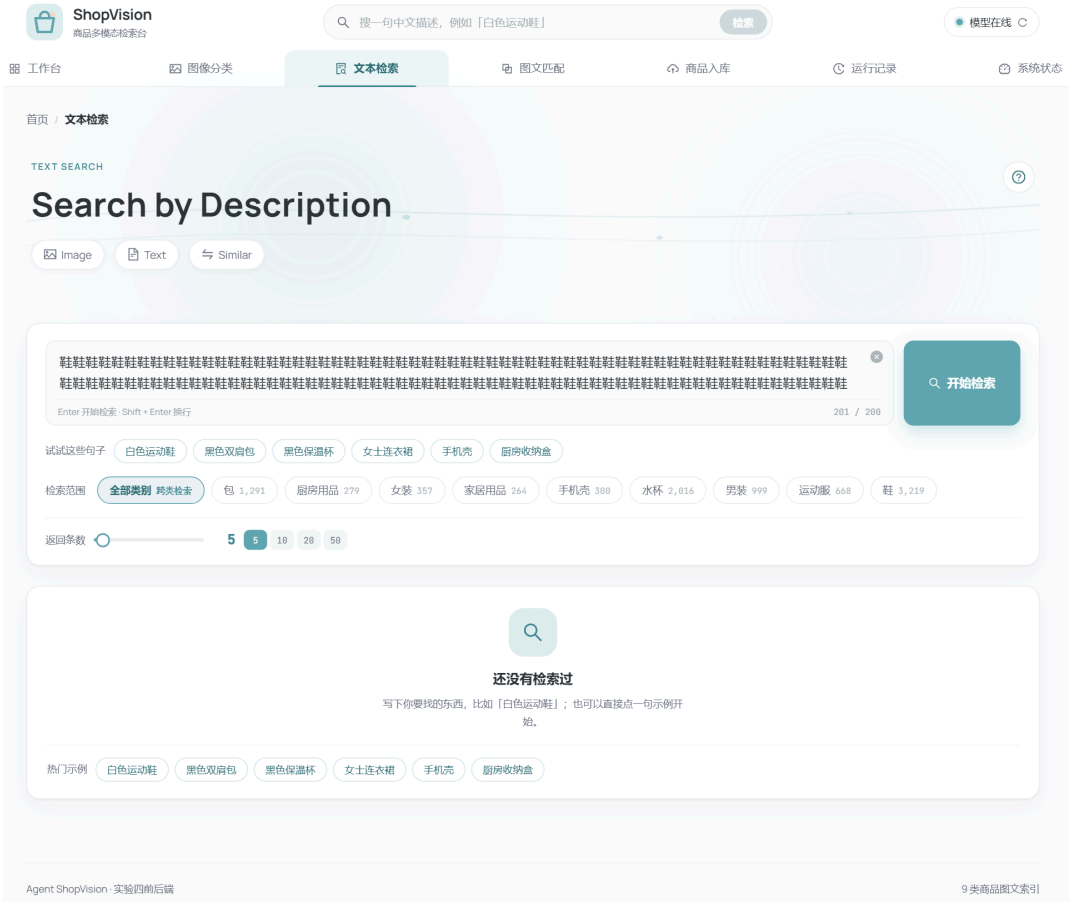


图 17 异常输入的中文错误提示

6.3 后端开发

FastAPI 提供 `POST /api/v1/agent/chat`、`POST /api/v1/agent/image` 和 `DELETE /api/v1/agent/session/{session_id}`。文本请求经过 Agent 决策、参数规范化、工具执行和结果点评两个阶段。图像请求直接使用实验四图像检索接口，并把结果写入会话历史。模型加载失败时回退到规则解析，工具失败时返回模板化提示。

6.4 前后端联调与测试

后端测试覆盖动作分流、参数校验、工具重试、结果安全闸、会话裁剪、摘要、槽位合并、图搜接话和 DELETE 路由。项目文档中的后端测试记录为 100 条，最新代码环境中实际完成的接口测试为 83 条，二者对应不同阶段的测试集合，报告采用最新可复现实测口径。前端代码和路由已经入库，最终交付前需要在安装 `node_modules` 后完成 npm test、npm run build 和浏览器截图回归。

第七章 大模型 Agent 部署

7.1 模型部署方案

本地模型服务使用 Qwen3-4B 加载 LoRA 适配器，提供 OpenAI 兼容的 `/v1/chat/completions`。LangChain Agent 通过统一模型接口调用本地服务，也可以将 `base_url` 指向局域网中的远程模型服务。商品事实仍由实验四检索 API 提供，模型输出仅作为结构化动作和自然语言说明。

7.2 模型选择与微调

Qwen3-4B 在现有 8GB 显存条件下可以使用 4-bit QLoRA。与全参数微调相比，QLoRA 只更新低秩适配器，显存和存储开销较小，适合本项目的工具调用和意图识别任务。训练数据从实验一和实验四的数据规范生成，包含属性识别、意图识别、工具调用和多轮对话样本。

7.3 Agent 框架集成

LangChain 负责模型与工具的编排，工具层只暴露白名单函数。Agent 输出动作后，后端检查工具名和参数，再调用检索服务。对 `text_search` 的查询词、类别和 `top_k` 进行规范化。结果回喂后，模型生成简短点评。模型不可用时由规则解析器生成等价动作，使系统仍能完成演示。

工具调用评测终端截图分别如图 图 18 和图 图 19 所示。第一张截图记录纯模型能力，第二张截图记录加入参数规范化后的实际运行效果。

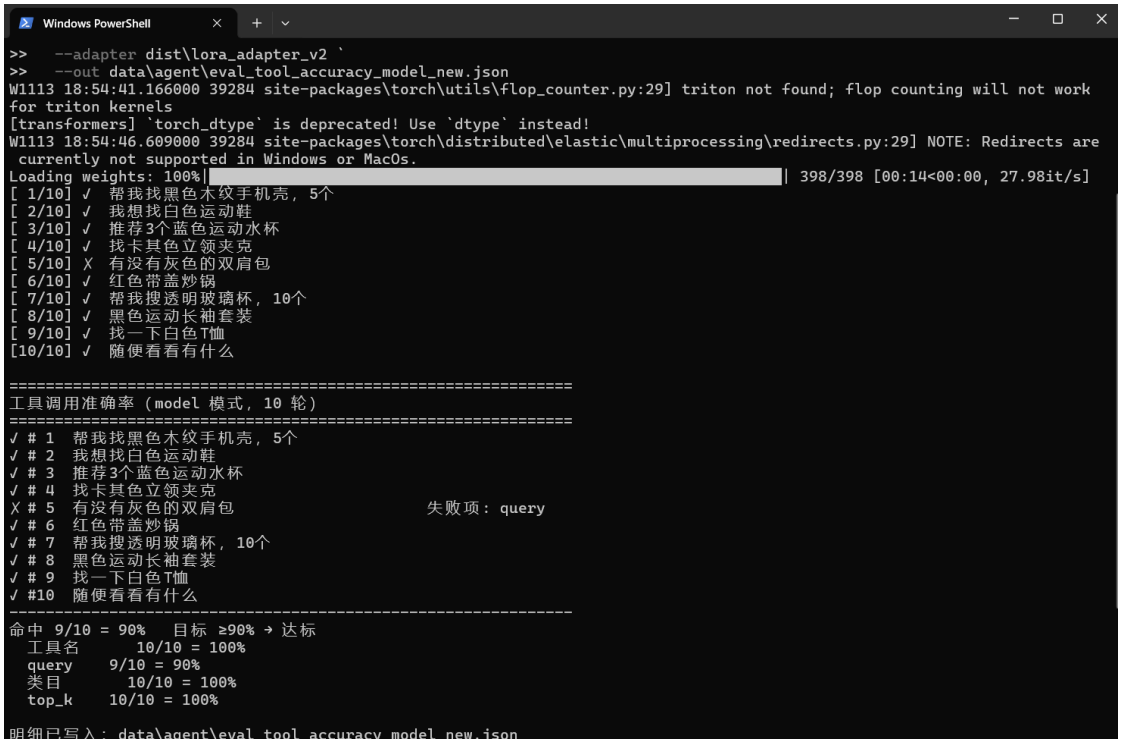


图 18 Agent 工具调用评测截图：纯模型 9/10 = 90%

纯模型评测命中 9/10，达到任务书要求的 90% 目标。

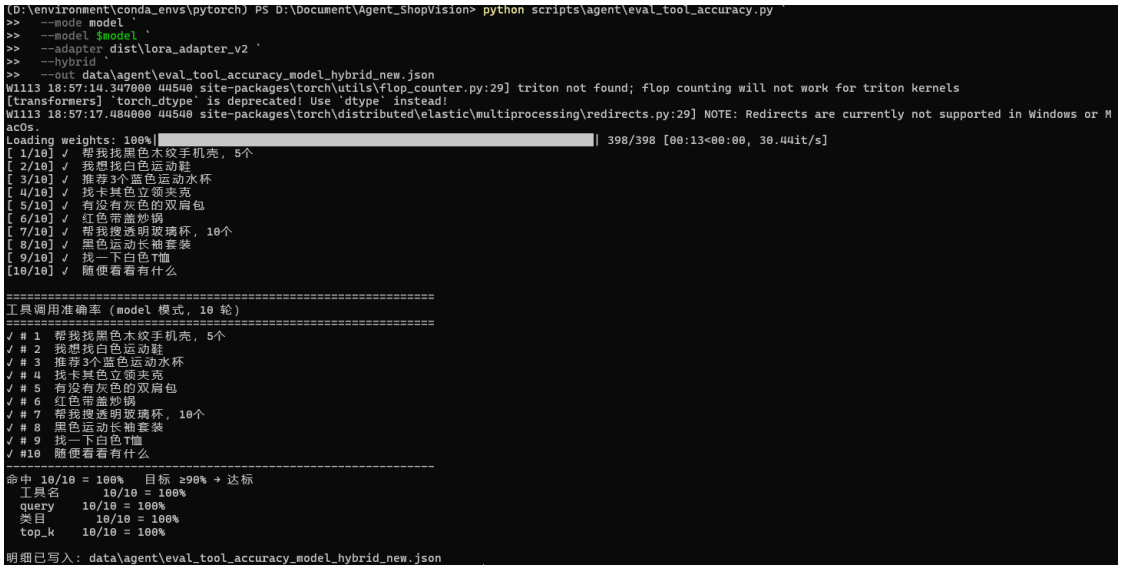


图 19 Agent 工具调用评测截图：混合模式 10/10 = 100%

混合模式在模型输出后增加查询词和类目规范化，命中 10/10。报告同时保留两种口径，避免把后处理带来的收益写成纯模型能力。

7.4 部署与测试

v2 在 RTX 3080Ti 与 RTX 5060 8GB 上结果一致，纯模型工具调用准确率均为 90%，加入后处理均为 100%，失败项为同一条“有没有灰色的双肩包”，差异来自查询词中的“的”。v4 多轮评测包含 40 条对话，JSON 解析率 100%，动作准确率 90%，检索命中率 82.5%，安全通过率 100%。这些结果说明模型已经能完成主要工具路由，但多轮参数和查询命中仍有优化空间。

前端集成回归的关键路径截图如图 图 20 至图 图 23 所示。需要说明的是，本轮回归在无 Qwen3-4B 基座权重的本机执行，Agent 走规则降级模式：图 图 20 验证首轮检索的商品卡片渲染链路；图 图 21 验证多轮追问的会话保持；图 图 22 验证图搜结果并入同一会话（图中首位相似度 0.9573 来自真实 ONNX 图像塔）；图 图 23 验证清空会话时前后端同步释放。say 过渡语与槽位继承的端到端效果需在装有 v4 模型的演示机上复测，清单已单列交付。

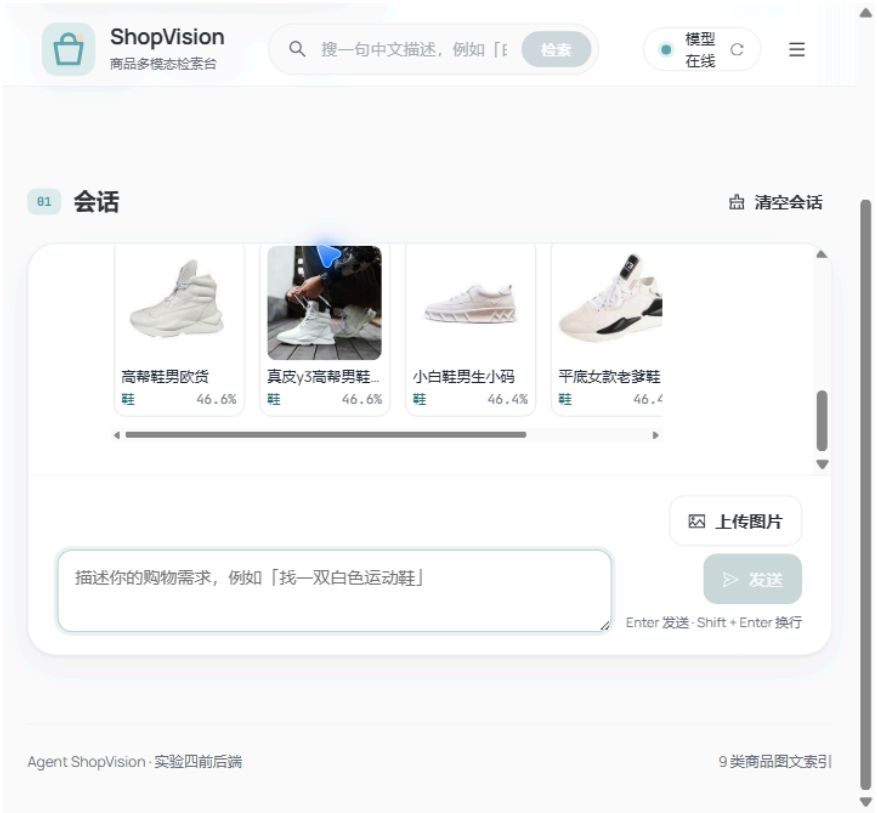


图 20 Agent 首轮检索：回复正文与商品卡片

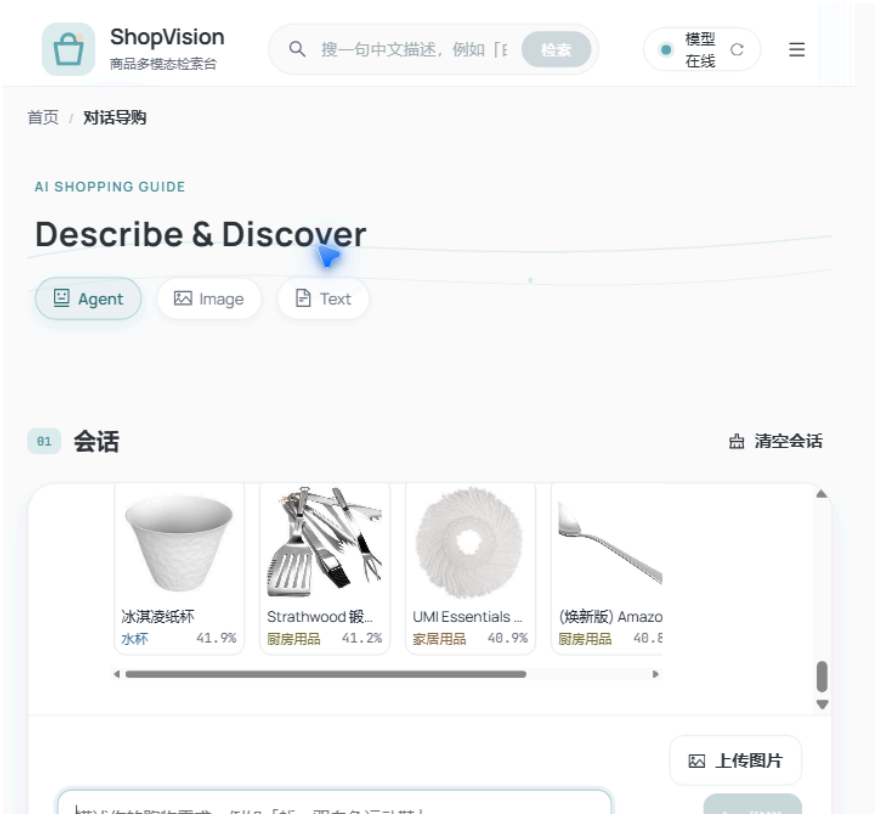


图 21 多轮追问：会话历史保持

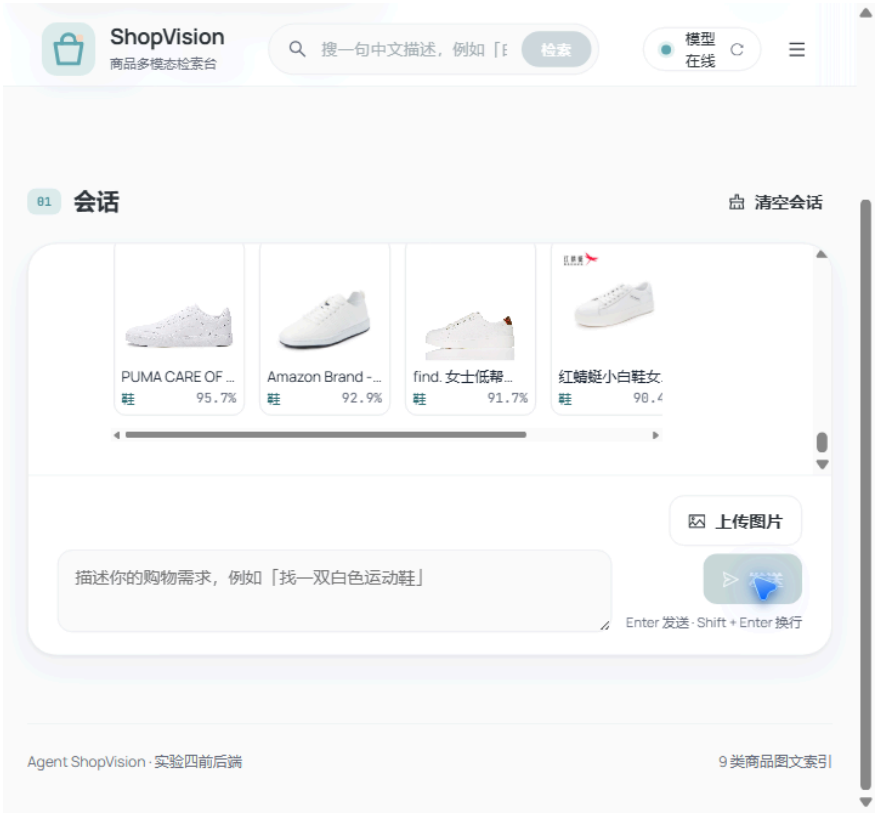


图 22 图搜接话：图像结果并入会话



图 23 清空会话：前后端同步释放

第八章 系统测试与评估

8.1 功能测试

功能测试覆盖文本检索、图片检索、图文匹配、Agent 对话、图片接话、会话清空和错误提示。文本检索通过 `category` 限定类别，图片检索先由图像塔编码再查询 FAISS，图文匹配分别判定两种模态后比较类别。Agent 的 `thinking` 字段用于前端展示调用过程。

8.2 性能测试

实验三的 TensorRT 指标来自同硬件配对测试。TensorRT 通过图优化和算子融合取得部署收益[8]。图像塔 batch=1 延迟降低 57.73%，batch=32 延迟降低 52.73%，吞吐分别提升 136.58% 和 111.52%。这些数字只代表图像塔 forward，不扩写为完整 HTTP 请求耗时。Agent 评测使用 10 轮和 40 条对话，重点考察工具动作和命中情况，不把小样本百分比当作泛化能力。

检索评测仪表盘如图 图 24 所示。

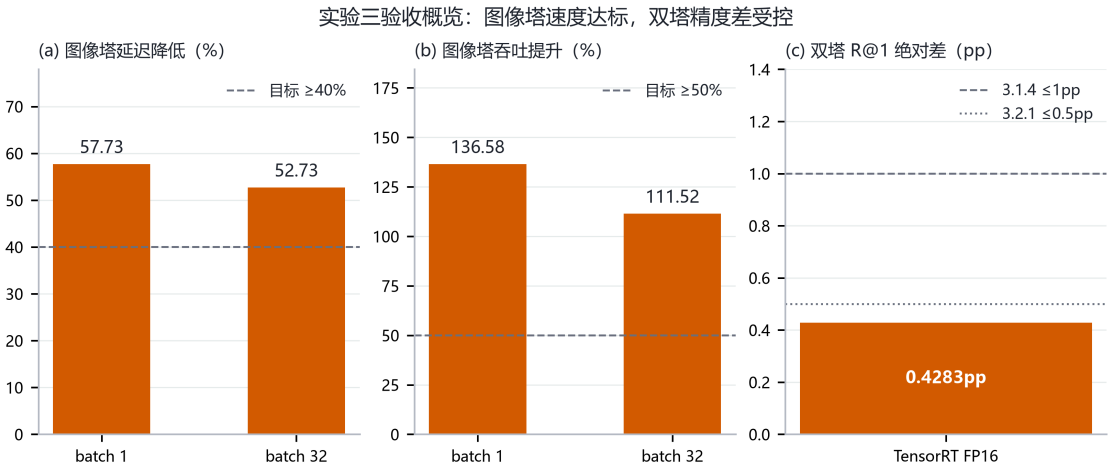


图 24 检索评测仪表盘

文本、图像和 Agent 的响应延迟记录如表 表 7 所示。

任务	平均响应时间	数据来源	状态
文本检索	均值 46.4 ms；P95 64.4 ms	本机端到端压测（ONNX CPU）	已完成
图片检索	均值 115.8 ms；P95 127.4 ms	本机端到端压测（ONNX CPU）	已完成
Agent 文本对话	均值 48.2 ms；P95 140.0 ms	本机端到端压测（规则降级 + 工具执行）	已完成
Agent 图片对话	均值 128.1 ms；P95 130.1 ms	本机端到端压测（ONNX CPU）	已完成
图像塔 TensorRT FP16	batch=1：1.94 ms；batch=32：23.84 ms	实验三同硬件测速	已完成

表 7 文本、图像与 Agent 响应延迟记录

表 表 7 的文本、图像与 Agent 四行均为本机端到端压测：在 Windows 11（Python 3.13，onnxruntime CPUExecutionProvider）上启动 FastAPI，检索后端为中国-CLIP ViT-B-16 双塔 ONNX fp32（9 类 9,393 条索引），四个接口各重复 30 次，取暖机后稳态的均值与 P95。模型在首次请求时一次性加载（双塔 ONNX 与 FAISS 索引），加载耗时不计入接口均值；表中数字均为加载完成后（暖机）的稳定耗时。Agent 文本对话在本机未加载 Qwen3-4B（缺 transformers 依赖与 GPU），决策走规则降级、工具执行仍为真实 ONNX 检索，故该行反映的是“规则决策 + 检索”完整链路；模型决策耗时以 GPU 演示机（RTX 5060）实测为准，见 5.4 节。

8.3 用户测试

用户侧测试按照演示流程进行。已有系统首页截图见图 14。第一步上传商品图，确认类别和相似商品卡片可显示。第二步输入“白色运动鞋”，确认文本结果来自鞋类索引。第三步上传鞋图并输入“双肩包”，确认图文匹配页面显示两类判定不一致。第四步在 Agent 页面输入“先找运动鞋，再要白色的”，确认第二轮继承上一轮类目并合并查询。正式交付前应补存 Chrome、Edge 和移动视口截图。

待补截图 1： Chrome、Edge、移动端三种视口下的 Agent 页面，需能看见输入框、工具调用过程和商品卡片。

待补截图 2：文本检索和图片检索各一次的端到端计时，截图中保留请求次数、平均耗时和模型后端。

现场演示采用十轮脚本，如表 表 8 所示。前九条都能对到 5.1 的 intent 与 tool_call 样本格式，第十条“随便看看有什么”不指定类目，用于验证规则降级路径。

轮次	输入	期望动作
1	帮我找黑色木纹手机壳，5 个	text_search / 手机壳 / 5
2	我想找白色运动鞋	text_search / 鞋 / 5
3	推荐 3 个蓝色运动水杯	text_search / 水杯 / 3
4	找卡其色立领夹克	text_search / 男装
5	有没有灰色的双肩包	text_search / 包
6	红色带盖炒锅	text_search / 厨房用品
7	帮我搜透明玻璃杯，10 个	text_search / 水杯 / 10
8	黑色运动长袖套装	text_search / 运动服
9	找一下白色 T 恤	text_search / 女装
10	随便看看有什么	text_search（不指定类目）

表 8 十轮演示脚本与期望动作

8.4 与预期目标对比分析

任务书目标与项目实测结果对比如表 表 9 所示。

目标	阈值	实测	结论
工具调用准确率	≥90%/10 轮	v2 纯模型 90%；后处理 100%	达到
TensorRT 延迟降低	≥40%	57.73%/52.73%	达到
TensorRT 吞吐提升	≥50%	136.58%/111.52%	达到
FP16 R@1 误差	≤1pp	0.4283pp	达到
前端完整回归	完成截图与 npm 测试	代码已入库，待最终回归	待补

表 9 任务书目标与项目实际结果

第九章 项目总结与展望

9.1 项目成果总结

项目完成了从数据集到可用 Agent 的端到端实现。数据集经过清洗和核验后支撑了 Chinese-CLIP 微调和 FAISS 索引。双塔模型在图文检索测试集上获得 58.67% 的 R@1。TensorRT FP16 达到任务书要求的推理收益。Qwen3-4B LoRA 模型能稳定输出主要工具调用，LangChain、FastAPI 和 React 已形成完整调用链。

9.2 技术创新点

项目的创新点体现在三个方面。第一，训练文本、索引文本和 Agent 结果使用统一商品描述模板，降低字段缺失造成的分布差异。第二，Agent 采用模型决策、工具执行、结果回喂和安全模板的两轮结构，商品事实由检索服务控制。第三，多轮会话使用槽位和查询合并处理“要白色的”等省略表达，并对工具名、类目和数量进行统一规范化。

9.3 遇到的挑战与解决方案

典型问题、原因和解决方案如表 表 10 所示。

问题	原因	解决方案
微调显存不足	4B 生成式模型和双塔模型显存占用不同，训练时需要保留梯度和优化器状态	采用 4-bit QLoRA、梯度检查点和单卡训练；适配器单独保存
Agent 工具调用失败	工具名拼写、数量词和类目表达不统一，初版 tool_call 样本偏少	扩充 v2 工具调用样本；增加白名单、编辑距离纠错、参数规范化和规则降级
前端跨域与端口冲突	前后端端口不同，PowerShell JSON 转义和已有进程会影响联调	使用 Vite 代理和统一 `/api/v1` 前缀；显式检查端口；使用 `ConvertTo-Json` 发送请求
多轮查询过于宽泛	模型只输出“白色”，没有继承上一轮“运动鞋”	保存会话槽位，合并同类目查询，并限制历史窗口

表 10 典型问题与解决方案

训练阶段显存有限，使用 QLoRA 和梯度检查点完成小模型微调。Agent 初版工具调用数据过少，出现工具名拼写错误，通过扩充样本重训和运行时白名单纠错解决。Windows 环境中曾出现端口占用、Python 启动器和 JSON 编码问题，通过 PowerShell `ConvertTo-`

Json、显式 Python 路径和端口检查完成排查。前端与后端接口需要统一会话字段，采用 localStorage 保存 session_id。

9.4 小模型与生成式模型微调对比

实验 2 与实验 5 的参数高效微调差异如表 表 11 所示。

对比项	实验 2 Chinese-CLIP	实验 5 Qwen3-4B Agent
目标	学习图像与商品文本的共享表示	学习意图、工具动作和多轮对话
微调方式	双塔对比学习，LoRA 权重用于图文编码	SFT + 4-bit QLoRA，更新生成式模型适配器
输入输出	图片/文本 → 512 维向量	用户话语 → JSON 动作与工具参数
评测	test 934 条，R@1 37.37% → 58.67%	10 轮工具调用纯模型 90%；v4 多轮动作 90%
关键优化	按 item_id 防泄漏，统一商品描述模板	增加工具调用样本，白名单纠错，槽位继承和安全闸

表 11 实验 2 与实验 5 参数高效微调对比

两类微调都没有改变基座模型的主体结构，区别在于任务目标和输出形态。实验 2 优化的是跨模态距离，实验 5 优化的是结构化动作生成。共同的关键点是控制数据分布、避免数据泄漏、使用较小学习率和有限训练轮数，并在部署侧保留可回退路径。不能用 Agent 的工具调用准确率替代图文检索 R@K，也不能把两类适配器混合加载。

9.5 项目局限性分析

5.2 验证集只有 11 条，tool_call 只有 1 条，因此 5.2 的 tool_call 百分比不具有统计代表性。当前 Agent 使用纯文本模型，图片理解由图像塔和检索服务完成。会话状态在进程内存中，重启服务后会丢失。检索对库外商品没有相似度阈值，可能返回 top-k 但不能据此判断库外商品。价格、品牌和材质字段存在缺失，系统不会在回复中编造这些属性。前端完整回归测试与最终浏览器截图仍需要补齐。

9.6 未来展望与改进方向

后续可以扩充人工审核的多轮对话数据，增加模糊查询、否定条件、价格区间和属性组合样本。可以把会话状态迁移到 Redis 或 SQLite，支持服务重启后恢复。可以增加检索相似度阈值和类别过滤，降低库外请求的误命中。前端完成三浏览器和移动端回归后，再将 Agent 服务部署到局域网，最后统一归档截图、视频、Word 和 PDF。

参考文献

[1] RADFORD A, KIM J W, HALLACY C, 等. Learning Transferable Visual Models From Natural Language Supervision[C/OL]//Proceedings of the 38th International Conference on Machine Learning (ICML). 2021: 8748-8763. https://proceedings.mlr.press/v139/radford21a.html.

[2] YANG A, PAN J, LIN J, 等. Chinese CLIP: Contrastive Vision-Language Pretraining in Chinese[EB/OL]. (2022). https://arxiv.org/abs/2211.01335.

[3] JOHNSON J, DOUZE M, JÉGOU H. Billion-scale Similarity Search with GPUs[J/OL]. IEEE Transactions on Big Data, 2021, 7(3): 535-547. https://arxiv.org/abs/1702.08734. DOI:10.1109/tbdata.2019.2921572.
[4] RAMÍREZ S. FastAPI Documentation[EB/OL]. (2026). https://fastapi.tiangolo.com/.
[5] META OPEN SOURCE. React Documentation: Quick Start[EB/OL]. (2026). https://react.dev/.
[6] HU E J, SHEN Y, WALLIS P, 等. LoRA: Low-Rank Adaptation of Large Language Models[C/OL]// International Conference on Learning Representations (ICLR). 2022. https://openreview.net/forum?id=nZeVKeeFYf9.
[7] ONNX OPEN SOURCE COMMUNITY. Open Neural Network Exchange (ONNX)[EB/OL]. (2026). https://onnx.ai/.
[8] NVIDIA CORPORATION. NVIDIA TensorRT: Best Practices[EB/OL]. (2026). https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/best-practices.html.

附录

附录 A 代码结构说明

项目主要目录和用途如表 表 12 所示。

目录	用途
`data/processed_data_cleaning/`	核验后的商品 CSV 与相对路径图片
`scripts/retrieval/`	Chinese-CLIP 编码、FAISS 索引和查询脚本
`scripts/agent/`	工具、意图解析器、会话状态、评测脚本
`webapp/backend/`	FastAPI、检索服务、Agent 接口和测试
`webapp/frontend/`	React 页面、AgentChat、Axios 与浏览器交互
`evaluation/retrieval/results/runs/`	实验三和检索评测的输入、表格、图片和说明
`docs/实验五/`	实验五阶段记录、交接文档和实跑结果

表 12 项目主要目录

附录 B 数据集详细统计

项目当前使用 9 个类别，清洗后的商品数为 9,393 条。训练集 7,520 条，验证集 939 条，测试集 934 条。每条记录保留 `item_id`、商品类别、标题、描述、颜色、材质、图片相对路径以及清洗状态字段。索引构建时额外生成 `vector_id` 和 `text`，便于 FAISS 结果回查。

附录 C 模型参数配置表

主要模型参数如表 表 13 所示。

模型	关键参数	值
Chinese-CLIP	输入/向量	224×224 / 512 维
LoRA	r / alpha / dropout	16 / 32 / 0.05
QLoRA	量化	NF4 + double quant，4-bit
Agent	训练轮数	3 epoch
FAISS	相似度	L2 归一化内积

表 13 主要模型参数

附录 D 系统部署文档

后端在 `webapp/backend` 下启动，前端在 `webapp/frontend` 下启动。Agent 模型服务使用 `scripts/agent/model_server.py`，默认提供 OpenAI 兼容接口。若使用局域网服务，服务端绑定 `0.0.0.0`，客户端将 `OPENAI_BASE_URL` 指向服务端的 `/v1`，并通过防火墙规则开放所需端口。正式环境需要补充鉴权和 HTTPS，开发演示使用局域网隔离。

附录 E 用户使用手册

1. 启动 FastAPI 后端并确认 `/api/v1/health` 返回正常。
2. 启动 React 前端并打开 `/agent` 页面。
3. 在输入框中输入商品查询，等待工具调用和结果卡片。
4. 图片查询使用商品图片上传区，系统返回类别和相似商品。
5. 多轮查询时保持同一个会话，不需要重复输入类别。
6. 遇到模型服务不可用时，系统会自动回退规则解析，仍可完成基本查询。